

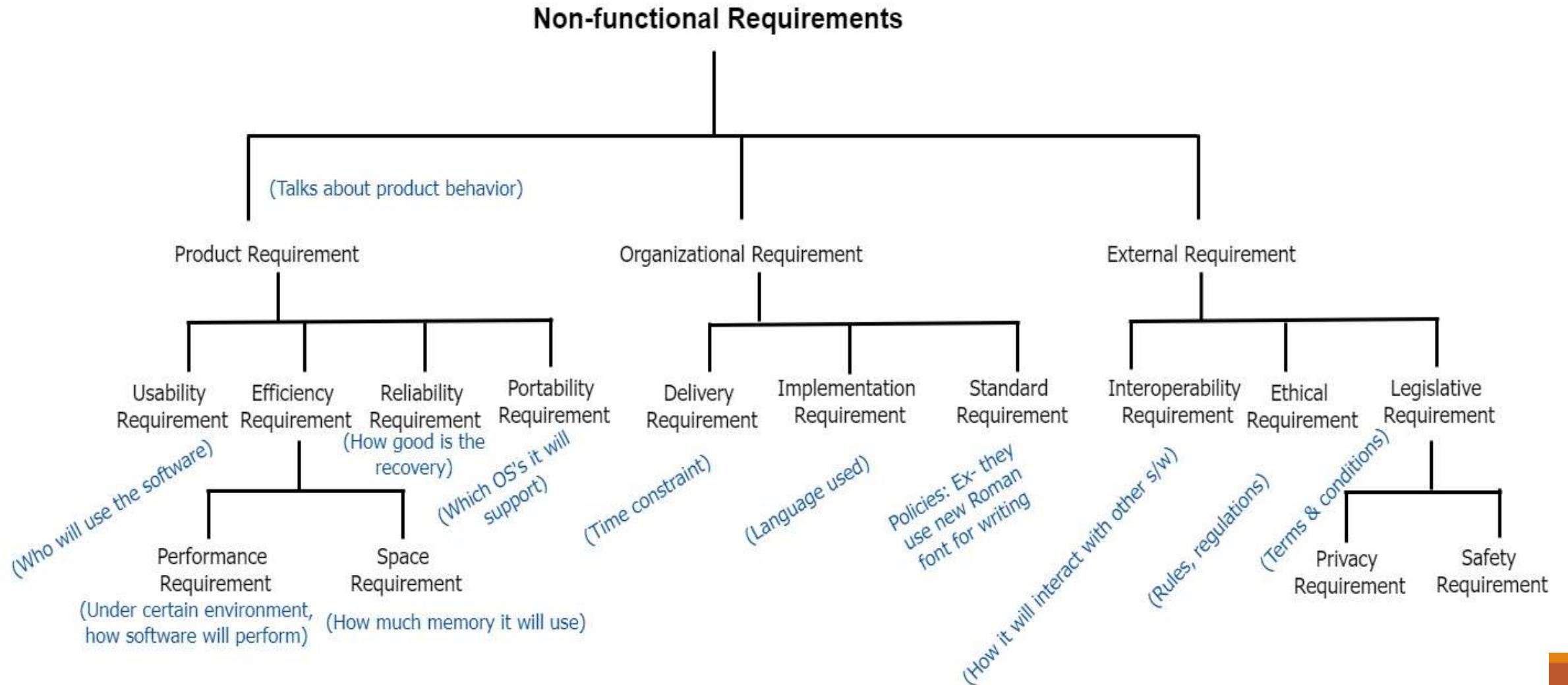
CSE327

Software Engineering

AKM IQTIDAR NEWAZ [IQN]

LECTURE - 5

Non-functional Requirements



Non-functional Requirements

The types of non-functional requirements are:

Product Requirements: These requirements specify product behavior. Examples include performance requirements on how fast the system must execute and how much memory it requires; reliability requirements that set out the acceptable failure rate; portability requirements; and usability requirements.

Organizational Requirements: These requirements are derived from policies and procedures in the customer's and developer's organization. Examples include process standards that must be used; implementation Requirements such as the programming language or design method used; and delivery requirements that specify when the product and its documentation are to be delivered.

Non-functional Requirements

External Requirements: This broad heading covers all requirements that are derived from factors external to the system and its development process. These may include interoperability requirements that define how the system interacts with systems in other organizations. There are other subcategories in legislative requirements that must be followed to ensure that the system operates within the law; and ethical requirements. Ethical requirements are requirements placed on a system to ensure that it will be acceptable to its users and the general public.

System Design

After requirement specification, we go for system design. Designing means:

Visualizing the internal system.

Blueprint of the system.

Deriving a solution.

Traced client's requirements.

Iterative process.

We start the design from high level overview, and it can be in object-oriented approach. Next, we will show how a use case diagram can be used to show the functionality of a system from the user perspective.

Use-Case Modeling

In use-case modeling, the system is looked upon as a black box whose boundaries are defined by its functionality to external stimuli.

The actual description of the use-case is usually given in plain text.

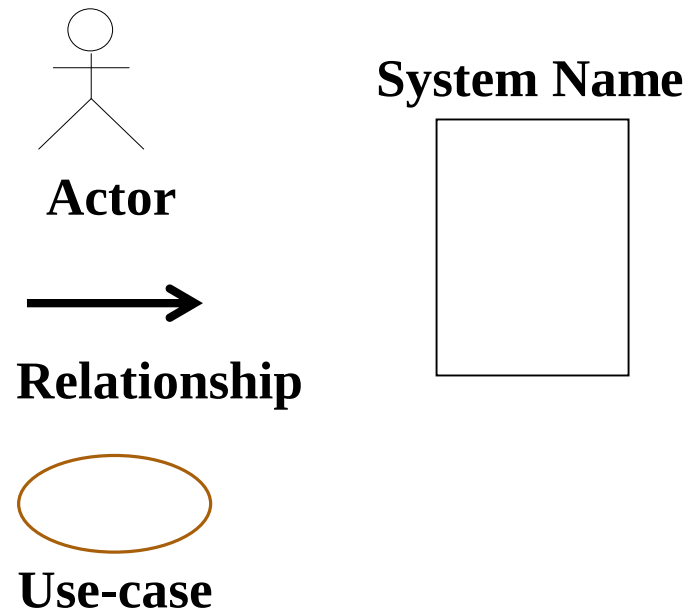
We will look into the details of text representation later. Both visual and text representation are needed for a complete view.

A use-case model represents the use-case view of the system. A use-case view of a system may consist of many Use-case diagrams.

A use-case diagram shows (the system), the actors, the use-cases and the relationship among them.

Components of Use-Case Model

- The components of a Use-case model are:
 - System Modeled
 - Actors
 - Use-cases
 - Stimulus



System

As a part of the use-case modeling, the boundaries of the system are developed.

System in the use-case diagram is a box with the name appearing on the top.

Define the scope of the system that you are going to design with your MyRo. (software scope)

MyRo Software Appln.



Actors

- An actor is something or someone that interacts with the system.
- Actor communicates with the system by sending and receiving messages.
- An actor provides the stimulus to activate an Use-case.
- Message sent by an actor may result in more messages to actors and to Use-cases.
- Actors can be ranked: primary and secondary; passive and active.
- Actor is a role not an individual instance.

Finding Actors

The actors of a system can be identified by answering a number of questions:

- Who will use the functionality of the system?
- Who will maintain the system?
- What devices does the system need to handle?
- What other system does this system need to interact?
- Who or what has interest in the results of this system?

Use-cases

A Use-case in UML is defined as a set of sequences of actions a system performs that yield an observable result of value to a particular actor.

Actions can involve communicating with number of actors as well as performing calculations and work inside the system.

A Use-case

- is always initiated by an actor.
- provides a value to an actor.
- must always be connected to at least one actor.
- must be a complete description.

Finding Use-cases

For each actor ask these questions:

- Which functions does the actor require from the system?
- What does the actor need to do?
- Could the actor's work be simplified or made efficient by new functions in the system?
- What events are needed in the system?
- What are the problems with the existing systems?
- What are the inputs and outputs of the system?

Describing Use-cases

Use-case Name:

Use-case Number: system#.diagram#.Use-case#

Authors:

Event(Stimulus):

Actors:

Overview: brief statement

Related Use-cases:

Typical Process description: Algorithm

Exceptions and how to handle exceptions:

Example

Number: A.132.4

Name: Buy book online

Author: B.Ramamurthy

Event: Customer request one or more books

System: Amazon.com

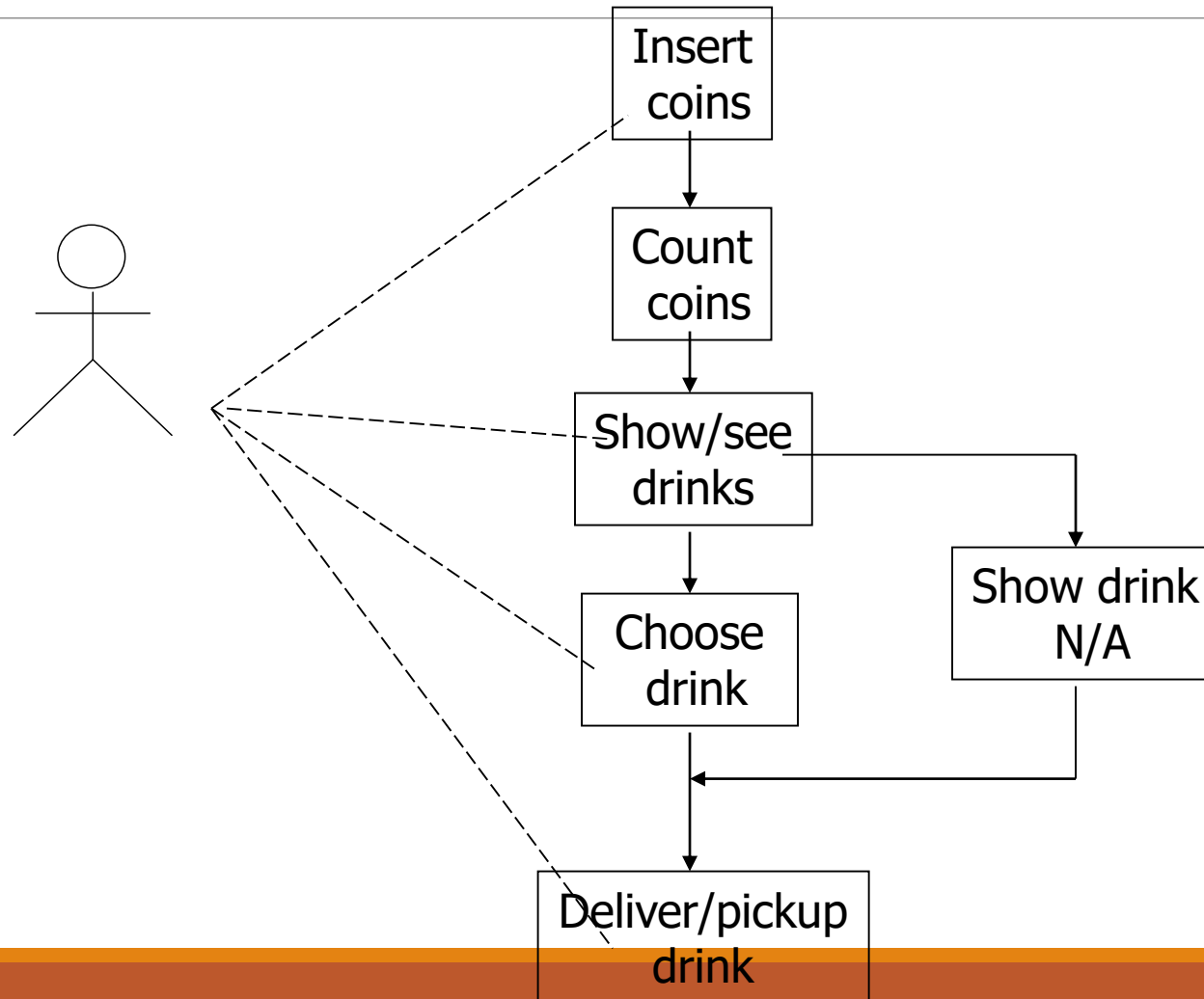
Overview: Captures the process of purchasing one or more books and the transactions associated with it.

Related Use-case: A.132.5, A.132.8

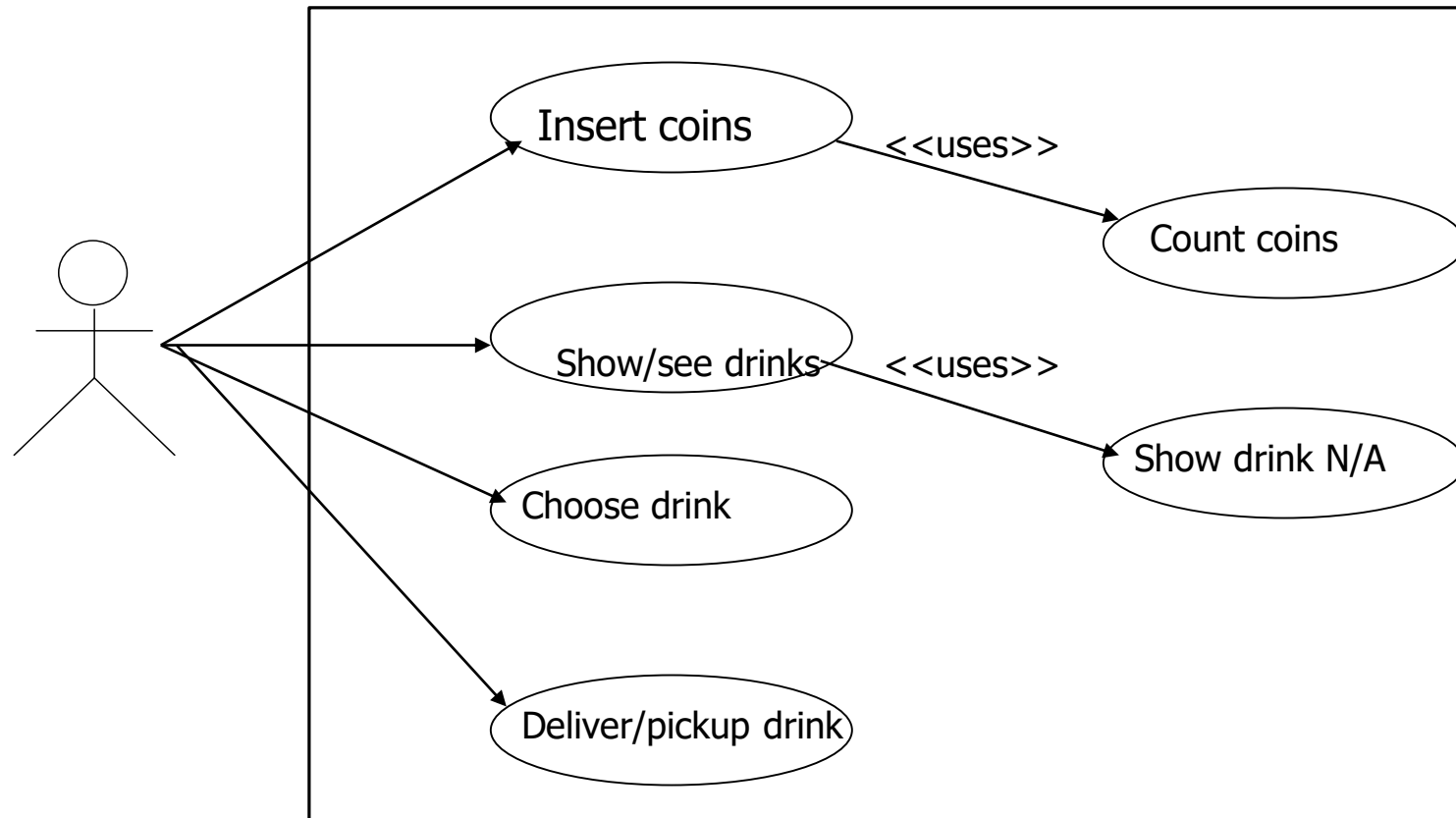
Typical Process Description with exceptions handled.

NOTE : All these can be in a tabular form, say, in an Excel worksheet for example.

Interaction between user and Use-case



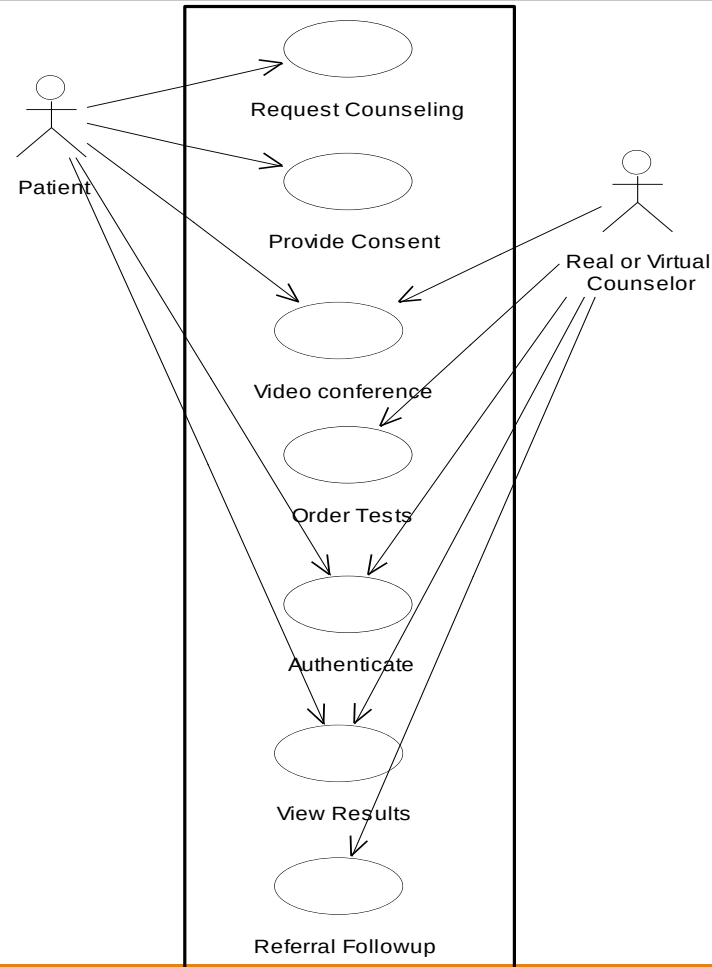
Use Case Diagram

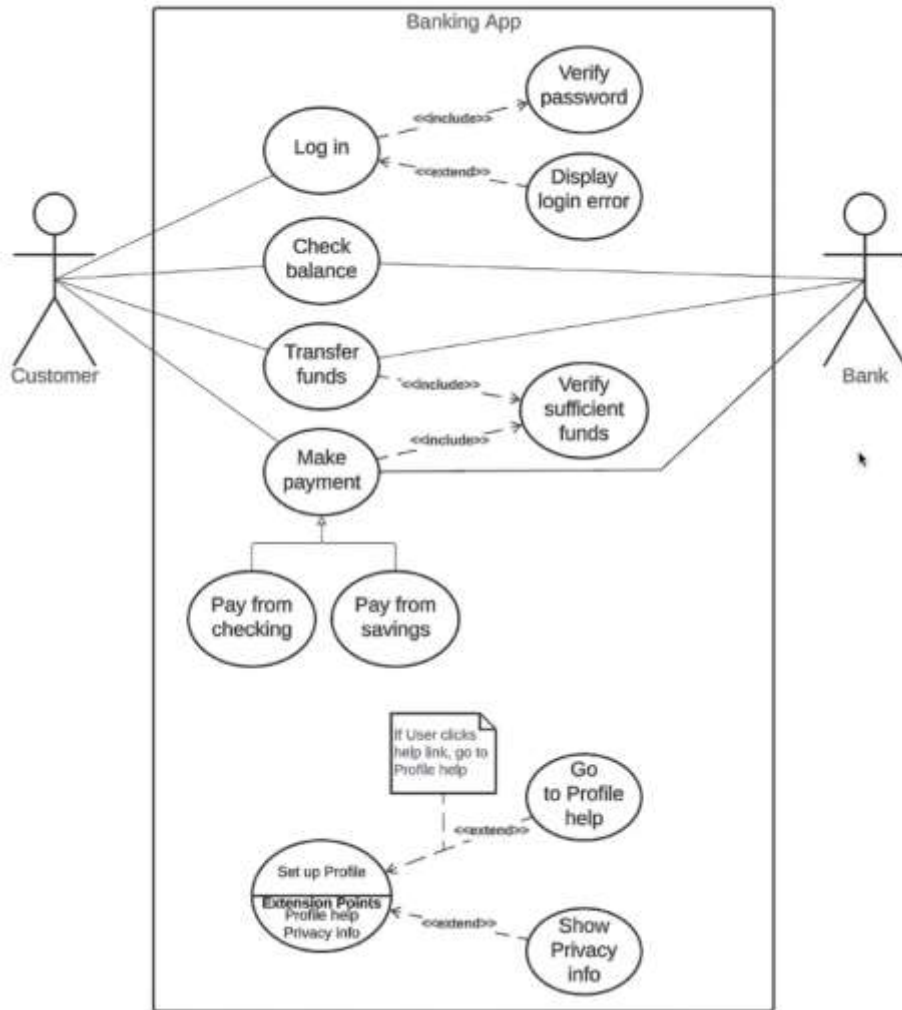


Case Studies

- Ticket counter for basketball game
- TIVO: Video recorder/controller
- Weather Station.
- ATM Machine: Description given as data dictionary.
- 4-cycle lawnmower engine
- Burger queen fast food restaurant's hand-held order device

Use Case Example: Counseling





Expanded Use Case – Banking App